

Overview

JIRA - [PrecisionPulse JIRA Board](#)

GitHub Repository - [PrecisionPulse Backend](#), [PrecisionPulse Frontend](#), [PrecisionPulse Desktop](#)

PrecisionPulse is a resilient, hybrid web-and-desktop ecosystem designed for real-time telemetry streaming and distributed data synchronization. The platform bridges a high-performance **Python desktop client** with a modern **Next.js web dashboard** using the **MQTT** protocol to ensure low-latency communication and robust offline capabilities.

Core Functionality

The system is built on three pillars: **Distributed Identity**, **Resilient Streaming**, and **Bi-directional Command Flow**.

1. Distributed Identity & Management

- **Decentralized Data:** There is no central database; credentials and configurations are mirrored in a **Web PostgreSQL** instance and a **Desktop SQLite** file.
- **Unified Auth:** Users access both platforms with the same credentials, with synchronization triggered via encrypted MQTT payloads whenever roles or permissions are modified.
- **Dynamic RBAC:** Administrators manage users and granular permissions from either interface, with changes propagating across the network in real-time.

2. Real-Time Telemetry "Share-Market" Style

- **Live Streams:** The desktop application periodically pushes telemetry data—including integers, decimals, and boolean flags—to the web dashboard.
- **Zero-Refresh UI:** Utilizing **WebSockets**, the web interface displays incoming data instantly without requiring page reloads, mimicking high-speed update terminals.
- **Connectivity Heartbeat:** Both ends feature a "live" status indicator; if the internet drops, the dashboard immediately reflects a "Disconnected" state.

3. Store-and-Forward Resilience

- **Offline Buffering:** If connectivity is lost at the client side, the desktop application automatically stores all outgoing records in its local SQLite database.
- **Automatic Flush:** Upon re-establishing a connection, the client prioritizes syncing all buffered historical data to the web application before resuming the live stream.

Business Impact

- **Operational Continuity:** Ensures zero data loss during network instability through the local buffering mechanism.
- **Scalability:** The lightweight MQTT protocol allows the system to handle multiple desktop clients streaming data simultaneously.

Flexibility: Admins can reconfigure system parameters (Key-Value pairs) on the fly from the web dashboard.

1. Functional Requirements

A. User & Access Management

- **Independent Credential Storage:** Both the Web (PostgreSQL) and Desktop (SQLite) applications must store user credentials locally.
- **Credential Mirroring:** Upon creating or updating a user at either end, the system must trigger an encrypted MQTT payload to update the corresponding record in the remote application's database.
- **Password Consistency:** Both applications must use the same encryption algorithm (e.g., Argon2) to ensure that a password hashed on the Web can be successfully verified on the Desktop.
- **Dynamic RBAC (Role-Based Access Control):** Admins must be able to create, update, and delete Users, Roles, and Permissions via either the web or desktop UI.
- **Bi-directional Sync:** Any change in User/Role/Permission objects at the web end must trigger an MQTT "Sync Command" to update the desktop client's local database immediately.

B. Real-Time Data Streaming & Control

- **Periodic Data Push:** The Desktop application must stream telemetry data (integers(double/long), decimals, and booleans) to the Web application at defined intervals without user intervention.
- **Live Dashboarding:** The Web UI must utilize WebSockets or MQTT-over-WebSockets to update numeric and status displays in real-time without page refreshes.
- **Zero-Refresh Updates:** The Web application must dynamically render incoming data streams from MQTT using a real-time bridge ( [Socket.IO](https://socket.io/)), ensuring values update as they arrive.
- **Remote Commands:** The Web application must be able to send control commands (e.g., "Force Sync," "Update Config") back to the Desktop application via specific MQTT topics.

- **Connection State Visibility:** Both UIs must feature a persistent status indicator (e.g., "Online" green pulse vs. "Offline - Syncing" amber icon).

C. Connectivity & Configuration

- **Connectivity Heartbeat:** Both applications must implement a "keep-alive" or heartbeat mechanism to display real-time "Connected/Disconnected" status indicators based on internet availability.
- **Dynamic Configuration:** The system must support Key-Value pair configurations that can be edited on the web and propagated to the desktop environment.
- **Local Persistence:** The Desktop application must utilize a local file-based database to ensure it remains functional even during periods of network outage.
- **Local Queuing (Offline Mode):** If connectivity is lost, the Desktop application must redirect the live data stream to a local SQLite "Buffer" table rather than attempting to send it over the network.
- **Automatic Synchronization:** Once a stable connection is re-established, the Desktop application must prioritize flushing the "Buffer" table to the Web application via MQTT before resuming live stream updates.

2. Non-Functional Requirements

- **Low Latency:** Data streamed from the desktop to the web must be displayed with sub-second latency to mimic "share price" style updates.
- **Security:** All MQTT communication must be encrypted via TLS/SSL, and sensitive password data must be hashed using industry-standard algorithms (e.g., Argon2 or BCrypt).
- **Reliability:** The synchronization logic must handle "conflict resolution" (e.g., identifying which update is more recent if data is changed on both ends simultaneously).

Portability: The web backend and database should be containerized to ensure consistent deployment across different environments.

3. Suggested Tech Stack

Component	Technology	Reasoning
Web Frontend	Next.js	Provides superior performance with Server-Side Rendering (SSR) and built-in API routes

		for frontend-backend integration.
Backend API	Python Flask	Lightweight and flexible for creating RESTful APIs and integrating with MQTT libraries like paho-mqtt.
Desktop Client	PyQt / PySide6	Allows for high-performance Python-based UI development with native access to the local file system.
Real-time Protocol	MQTT (EMQX/Mosquitto)	The standard protocol for lightweight, bi-directional communication between remote clients and servers.
Primary Database	PostgreSQL	Robust relational database to handle central users, roles, and historical data logs.
Local Database	SQLite	A self-contained, serverless file database ideal for local desktop persistence.
Real-time Bridge	Socket IO / WebSockets	To bridge MQTT messages coming into the Flask backend directly to the Next.js frontend.
Testing & Quality	Pytest & Sonar	Ensures logic for synchronization and data streaming is bug-free and follows high code standards.

4. Proposed Sync Workflow

1. **Normal State:** Desktop streams data → MQTT Broker → Flask Backend → Web Dashboard.
2. **Disconnected State:** Desktop detects broker loss → Starts writing data to local_buffer table in SQLite.
3. **Reconnection:** Desktop detects broker → Fetches records from local_buffer → Publishes to MQTT → Deletes from SQLite upon confirmation.